UNIVERSITY GRADE & ATTENDANCE SYSTEM

JAVA PROGRAMMING PROJECT • MODULES 1–6

PROJECT TEAM

M. Haasini — 2620030081 | Aviyah — 2620030112
Course: PSPJ | Section: 12 | Faculty: A. Swathi

TRACK ATTENDANCE • CALCULATE GRADES • ORGANIZE STUDENT RECORDS

ABSTRACT

Project overview and core process

PROJECT OVERVIEW

A menu-driven Java console application that manages student records, marks and attendance.
It calculates each student's average and letter grade, checks the 75% attendance requirement, searches and sorts records, and displays a class summary.
Records are saved to a text file so they can be loaded when the program starts.

INPUT → PROCESS → OUTPUT

INPUT
Roll number, name, marks for 3 subjects, attendance percentage
PROCESS
Validate entries, calculate average and grade, check attendance eligibility
OUTPUT
Student details, attendance-shortage list, sorted list, class summary and saved records

UNDERSTANDING THE PROBLEM STATEMENT

Why a small academic record tracker is useful

CHALLENGES

- Manual calculations can cause mistakes
- Finding one student's record can take time
- Attendance shortage may go unnoticed
- Records can be lost between sessions

PROPOSED SOLUTION

- One menu for common student-record tasks
- Automatic average and grade calculation
- Search by roll number
- Flag attendance below 75%
- Sort students by average
- Save and reload records

SYSTEM WORKFLOW & FEATURES

The workflow reflects the actual program menu

1 Load records

Read students.txt if it exists

2 Show menu

Repeat until Save and Exit

3 Process choice

Add, display, search, shortage, sort or summary

4 Save records

Write student data to students.txt

KEY FEATURES

Add student • Display all • Search roll number • Attendance shortage • Sort by average • Class summary

VALIDATION

Reject duplicate roll numbers, invalid marks/attendance, and empty or comma-containing names.

CO 1 — MODULE 1: JAVA FOUNDATIONS

Concepts shown in the actual source code

CONCEPTS USED

- Variables and primitive data types: int, double
- Constant: MIN_ATTENDANCE = 75.0
- Scanner for console input
- System.out.print / println / printf
- Integer.parseInt() and Double.parseDouble()
- Breaking a problem into smaller tasks

EXAMPLES FROM THE CODE

```
static Scanner sc = new Scanner(System.in);

static final double MIN_ATTENDANCE = 75.0;

int roll = readInt();
double attendance = readDouble();

System.out.printf("Class average: %.2f%n",
classAverage);
```

CO 2 — MODULE 2: CONTROL FLOW

Selection, repetition and validation

CONCEPTS USED

- if / else and else-if conditions
- switch with case and default
- do-while menu loop
- while(true) input-retry loops
- for and enhanced for loops
- break and return
- Input range checks

EXAMPLES FROM THE CODE

```
do {
   // display menu and read choice
   switch (choice) {
      case 1: addStudent(); break;
      case 2: displayStudents(); break;
      default: System.out.println("Invalid choice.");
   }
} while (choice != 7);

if (attendance < 0 || attendance > 100) {
   throw new IllegalArgumentException(...);
}
```

CO 3 — MODULE 3: METHODS & ARRAYS

Reusable operations, searching and calculations

CONCEPTS USED

- Methods with parameters and return values
- One-dimensional int[] array for subject marks
- Array traversal with for / for-each
- Summation and average calculation
- Searching records by roll number
- Returning null when a student is not found

EXAMPLES FROM THE CODE

```
int[] marks = new int[3];

static Student findStudent(int roll) {
    for (Student s : students) {
        if (s.getRollNo() == roll) return s;
    }
    return null;
}

double getAverage() {
    int total = 0;
    for (int mark : marks) total += mark;
    return (double) total / marks.length;
}
```

CO 4 – MODULE 4: OOP & EXCEPTION HANDLING

Only the object-oriented and exception concepts present in the code

CONCEPTS USED

- Student class and Student objects
- Parameterized constructor
- Private fields (encapsulation)
- Getter methods
- Overriding toString()
- try-catch exception handling
- IllegalArgumentException, NumberFormatException and IOException
- try-with-resources while saving

EXAMPLES FROM THE CODE

```
static class Student {
    private int rollNo;
    private String name;
    private int[] marks;
    private double attendance;

    Student(int rollNo, String name,
            int[] marks, double attendance) {
        this.rollNo = rollNo;
        this.name = name;
        this.marks = Arrays.copyOf(marks, marks.length);
        this.attendance = attendance;
    }
}
```

CO 5 — MODULE 5: STRINGS & FILE HANDLING

Text processing and persistent records

CONCEPTS USED

- String operations: trim(), isEmpty(), contains(), split()
- String concatenation to create a record line
- Parsing text values into numbers
- Path and Files for file access
- Reading records from students.txt
- Writing records with PrintWriter
- Arrays.copyOf() and Arrays.toString()

EXAMPLES FROM THE CODE

```
String name = sc.nextLine().trim();

String toFileLine() {
    return rollNo + "," + name + "," + marks[0] + ","
        + marks[1] + "," + marks[2] + "," + attendance;
}

Path path = Path.of(FILE_NAME);
if (Files.exists(path)) {
    for (String line : Files.readAllLines(path)) {
        // parse each saved record
    }
}
```

CO 6 — MODULE 6: COLLECTIONS & STREAMS

Dynamic records, sorting and stream-based summaries

CONCEPTS USED

- ArrayList<Student> and List<Student>
- Adding records dynamically
- Comparator.comparingDouble() for sorting
- Streams and lambda filter()
- collect(Collectors.toList())
- mapToDouble() and average()
- Optional.orElse() fallback
- count(), forEach() and method references

EXAMPLES FROM THE CODE

```
static ArrayList<Student> students = new ArrayList<>();

List<Student> shortageList = students.stream()
    .filter(s -> s.getAttendance() < MIN_ATTENDANCE)
    .collect(Collectors.toList());

List<Student> sorted = new ArrayList<>(students);
sorted.sort(Comparator.comparingDouble(
    Student::getAverage).reversed());
```

CODE SCREENSHOT 1 — MAIN PROGRAM & MENU

```
t java.io.*;
t java.util.*;

c class UniversityGradeAttendanceSystem {
tatic Scanner sc = new Scanner(System.in);
tatic ArrayList<Student> students = new ArrayList<>();

un | Debug
ublic static void main(String[] args) {
    int choice;
    do {
        System.out.println(x: "\n1. Add Student  2. Display  3. Search  4. Save
        choice = readInt(field: "choice");

        switch (choice) {
            case 1: addStudent(); break;
            case 2: displayStudents(); break;
            case 3: searchStudent(); break;
            case 4: saveData(); break;
            default: System.out.println(x: "Invalid choice. Try again.");
        }
    } while (choice != 4);


/ Keeps asking until the user enters a valid whole number.
tatic int readInt(String field) {
    while (true) {
        System.out.print("Enter " + field + ": ");
        String text = sc.nextLine().trim();
        if (text.isEmpty()) {
            System.out.println(x: "Input cannot be empty. Please try again.");
            continue;
        }
        try {
            return Integer.parseInt(text);
        } catch (NumberFormatException e) {
            System.out.println(x: "Invalid input. Enter a whole number.");
        }
    }
}

static double readDouble(String field) {
    while (true) {
        System.out.print("Enter " + field + ": ");
        String text = sc.nextLine().trim();
        if (text.isEmpty()) {
            System.out.println(x: "Input cannot be empty. Please try again
            continue;
        }
        try {
            return Double.parseDouble(text);
        } catch (NumberFormatException e) {
            System.out.println(x: "Invalid input. Enter a number.");
        }
    }
}

// Modules 1 and 2: input, output, conditions and loops
static void addStudent() {
    int roll = readInt(field: "roll number");

    System.out.print(s: "Enter student name: ");
    String name = sc.nextLine().trim();
```

CODE SCREENSHOT 2 — MAIN PROGRAM & MENU

```
        while (name.isEmpty()) {
            System.out.println(x: "Name cannot be empty.");
            System.out.print(s: "Enter student name: ");
            name = sc.nextLine().trim();
        }

        int[] marks = new int[3];
        for (int i = 0; i < marks.length; i++) {
            while (true) {
                marks[i] = readInt("marks for subject " + (i + 1) + " (0-100
                if (marks[i] >= 0 && marks[i] <= 100) break;
                System.out.println(x: "Marks must be between 0 and 100.");
            }
        }

        double attendance;
        while (true) {
            attendance = readDouble(field: "attendance percentage (0-100)");
            if (attendance >= 0 && attendance <= 100) break;
            System.out.println(x: "Attendance must be between 0 and 100.");
        }

        students.add(new Student(roll, name, marks, attendance));
        System.out.println(x: "Student added successfully.");
    }

    static void displayStudents() {
        if (students.isEmpty()) {
            System.out.println(x: "No students have been added yet.");
            return;
        }
```

```
90      static void displayStudents() {
95          for (Student s : students) System.out.println(s);
96      }
97
98      static void searchStudent() {
99          int roll = readInt(field: "roll number to search");
00          for (Student s : students) {
01              if (s.roll == roll) {
02                  System.out.println(s);
03                  return;
04              }
05          }
06          System.out.println(x: "Student not found.");
07      }
08
09      // Module 5: file writing and exception handling
10      static void saveData() {
11          try (PrintWriter out = new PrintWriter(fileName: "students.txt")) {
12              for (Student s : students) {
13                  out.println(s.roll + "," + s.name + "," + s.marks[0] + ","
14                          + s.marks[1] + "," + s.marks[2] + "," + s.attendance)
15              }
16              System.out.println(x: "Saved to students.txt");
17          } catch (IOException e) {
18              System.out.println(x: "Could not save file.");
19          }
20      }
21
22      // Module 4: class, object, constructor and method overriding
23      static class Student {
24          int roll;
25          String name;
```

CODE SCREENSHOT 3 — MAIN PROGRAM & MENU

```
126         int[] marks;
127         double attendance;
128
129         Student(int roll, String name, int[] marks, double attendance) {
130             this.roll = roll;
131             this.name = name;
132             this.marks = marks;
133             this.attendance = attendance;
134         }
135
136         // Module 3: method and array traversal
137         double average() {
138             int total = 0;
139             for (int mark : marks) total += mark;
140             return (double) total / marks.length;
141         }
142
143         char grade() {
144             double avg = average();
145             if (avg >= 90) return 'A';
146             if (avg >= 75) return 'B';
147             if (avg >= 60) return 'C';
148             if (avg >= 40) return 'D';
149             return 'F';
150         }
151
152         // Module 6: stream and lambda
153         String marksText() {
154             return Arrays.stream(marks)
155                     .mapToObj(String::valueOf)
156                     .reduce((a, b) -> a + ", " + b).orElse(other: "");
```

```
n Arrays.stream(marks)
    .mapToObj(String::valueOf)
    .reduce((a, b) -> a + ", " + b).orElse

ring toString() {
n "Roll: " + roll + " | Name: " + name
    + " | Marks: " + marksText()
    + " | Average: " + String.format(forma
    + " | Grade: " + grade()
    + " | Attendance: " + attendance + "%"
```

SAMPLE PROGRAM OUTPUT

Illustrative output aligned with the actual program's fields and menu

```
 Display  3. Search  4. Save & Exit

 23
: tanya
bject 1 (0-100): 87
bject 2 (0-100): 97
bject 3 (0-100): 79
ercentage (0-100): 89
essfully.

 Display  3. Search  4. Save & Exit

anya | Marks: 87, 97, 79 | Average: 87.67 | Grade: B | A

 Display  3. Search  4. Save & Exit
```

```
 3. Search  4. Save & Exit


: 23
rks: 87, 97, 79 | Average: 87.67 | Gra

 3. Search  4. Save & Exit
```

CONCLUSION & FUTURE SCOPE

\Summary of the implementation

CONCLUSION

The project applies Java concepts from Modules 1–6 in a single console application.

It can add, display, search, sort and summarize student records; calculate averages and grades; identify attendance below 75%; and save records to a text file.

FUTURE SCOPE

- Database storage instead of a text file
- A graphical interface
- Exportable reports
- Separate roles and logins
- Automated attendance notifications

GEO-TAG PHOTO & GITHUB REPOSITORY

Complete these items before final submission

GEO-TAGGED PHOTO

GITHUB REPOSITORY

THANK YOU